Team 10 · IoT Mini Project · UE23CS342AA3

FlowState

IoT water leak detection and consumption analytics using dual flow sensors

[Team member names]

Sense → Ingest → Process → Store → Analyze

Hidden leaks waste water every day

35,000 L+

wasted a year by the leaks in an average home

US EPA WaterSense

11,000 L+

a year from one tap dripping once a second

US EPA WaterSense

Worst

water crisis in India’s history, NITI Aayog warned in 2018

NITI Aayog, via The Wire

A meter shows how much was used, not how much was lost. **Two sensors on one pipe show the difference within a minute.**

Analysis first: five questions

What FlowState answers

1. Am I losing water, and how much?
2. When do I use the most water?
3. Is today’s use unusual?
4. Is water flowing at night?
5. What does it cost?

1 **Sense**
Two flow sensors, tank level and humidity, every second

2 **Ingest**
MQTT every 2 s for live data, HTTPS every 10 s for storage

3 **Process**
On the ESP32: litres, a 60 s leak check, automatic shut-off

4 **Store**
Supabase (PostgreSQL), every value checked

5 **Analyze**
The dashboard answers the five questions

Architecture: edge first

Sense · sensors and actuators
2 × YF-S201 flow (GPIO 19, 22) · level (ADC 33) · DHT22 · DS3231 (I²C) · SG90 valve (PWM) · buzzer

Process · ESP32 DevKit V1, at the edge
Pulses → litres every second · 60 s leak check · night watch 2–5 AM · learns normal use · closes the valve

MQTT over TLS · every 2 s · commands back

HTTPS POST · every 10 s

Ingest · MQTT broker
Telemetry · status with Last Will · alerts · signed commands · ping

Store · Supabase PostgreSQL
Every reading, range-checked · live stream · hourly summary

MQTT over WebSocket

Realtime + SQL

Analyze · web dashboard
Live view · five questions · alerts · performance panel · valve control

Hardware: about ₹2,500–3,500

Part	Job	Pin
YF-S201 flow sensor 1	Tank outlet	GPIO 19
YF-S201 flow sensor 2	Tap	GPIO 22
Capacitive level sensor	Tank level	ADC 33
DHT22	Humidity, temperature	GPIO 4
DS3231 clock	Time without Wi-Fi	I²C 18/21
SG90 servo + ball valve	Shut-off valve	PWM 26
Active buzzer	Alarm	GPIO 13

5 V pulses, 3.3 V pins

Each flow signal passes a divider: 10 kΩ to the pin, 20 kΩ to ground.

Separate servo supply

The SG90 runs from its own 5 V; all grounds are joined.

Calibrated

Bucket test for ~450 pulses per litre; empty and full readings for the level.

The ESP32 decides on its own

`loss = (S1 − S2) ÷ S1, every 60 s`

Normal · under 5%

Warning · 5–15%

Critical · 15% and above: valve closes, buzzer sounds

Night watch, 2–5 AM

5 minutes of flow or 0.5 L raises an alert; 10 minutes closes the valve.

Learns normal use

Three days of hourly averages, then use above 1.5× normal is flagged.

Works offline

Decisions need no Wi-Fi; the daily total survives a power cut.

`One file per stage: sense.ino · process.ino · cloud.ino · mqtt.ino · console.ino`

MQTT for live, HTTPS to store

	MQTT	HTTPS (REST)
Job	Live view, status, valve commands	Every reading stored for analysis
How often	Every 2 s	Every 10 s
Connection	One TLS link stays open; pushes both ways	A request per reading; the TLS link is reused
Delivery check	Sequence numbers count lost messages	Readings stored against one every 10 s
If it fails	Last Will tells the dashboard “offline”	Retried once; failures are counted
Measured	310 ms round trip, 100% delivered, 220 bytes	Upload time reported by the ESP32

`Topics under flowstate/team10-d76014: /telemetry · /status (retained, Last Will) · /alert · /cmd (QoS 1) · /ack · /ping and /pong`

Safe valve control over MQTT

1

Dashboard signs

HMAC-SHA256 with the valve key over the action, the time and a one-time code

2

Broker delivers

QoS 1 on /cmd; anyone may publish there, so only the signature counts

3

ESP32 checks

Right signature, under 5 minutes old, newer than the last command

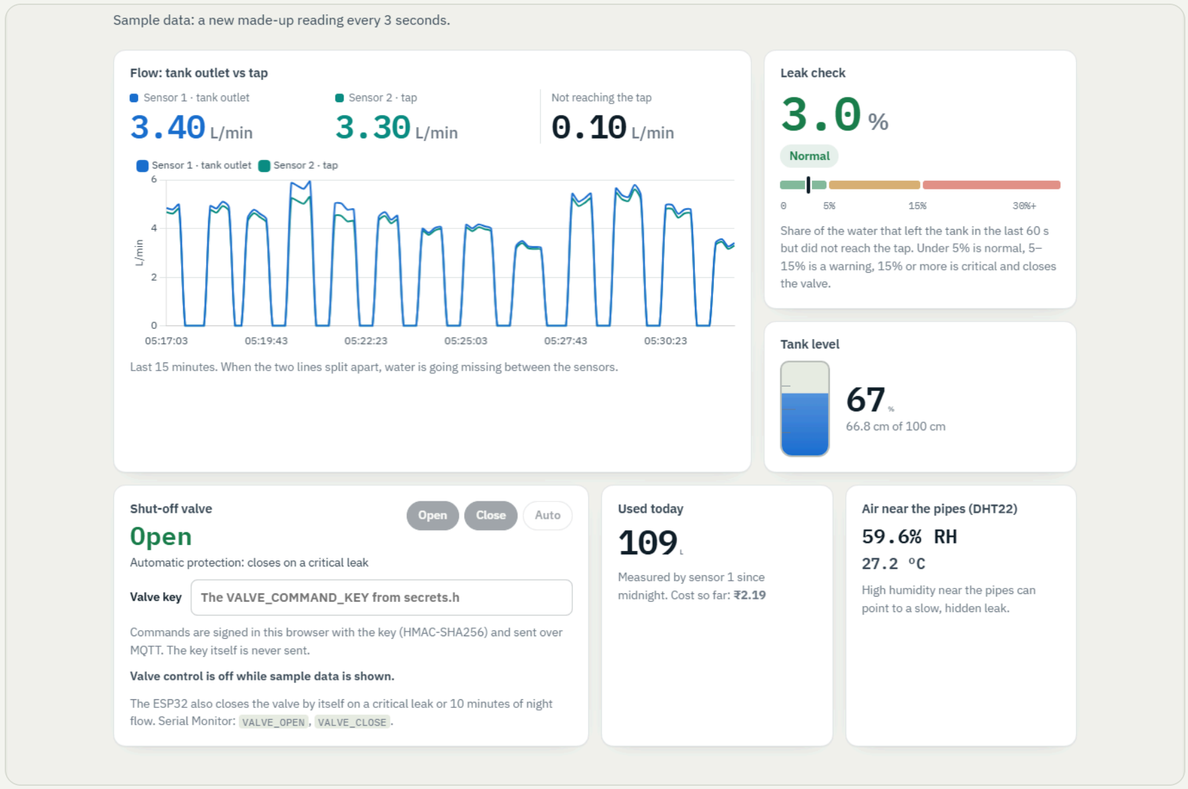
4

Valve moves

The servo turns and an answer comes back: 1.24 s measured

The key never leaves the browser. **KEY_CHECK** on the Serial Monitor prints the same 8-character fingerprint the dashboard shows.

The dashboard, live



Shown with sample data, labelled as such on the page

Values change as readings arrive

Every 2 s over MQTT; each new value flashes

The pipeline lights up

Sense to Analyze, stage by stage, for every reading

Two lines that split mean a leak

Sensor 1 and sensor 2 drawn together for 15 minutes

Phone, light and dark

sustainflow.netlify.app

Five questions, five answers

Question	How it is worked out	Shown as
Am I losing water?	Lost litres per day and in 7 days; under 5% is sensor error	Bars: reached the tap, lost
When do I use the most?	Average litres in each hour of the day	Bars by hour, today as a line
Is today unusual?	Today so far against previous days by the same time; 1.5× is unusual	Today against a normal day
Is water flowing at night?	Minutes of flow between 2 and 5 AM, each night	Seven nights: dry, some, alert
What does it cost?	Litres × tariff (₹ per 1,000 L, editable)	Today, a month, leaks a month

The database sums each hour (get_hourly_summary), so a week of analysis loads in one small request.

Tested end to end

141/141

automated checks pass: firmware 61, database 42, dashboard 26, browser 12

310 ms

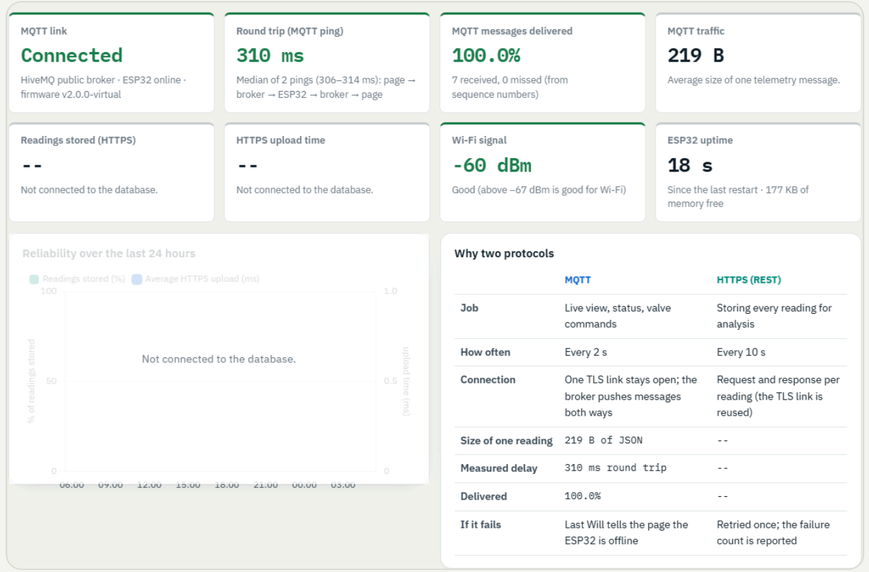
MQTT round trip, page to device and back (median)

100%

MQTT messages delivered, counted from sequence numbers

1.24 s

signed Close command, click to confirmation



Measured on the real HiveMQ broker with a stand-in device, 12 Nov 2026

In the lab: hardware tests H1–H10, including calibration, a leak simulation, the night watch, network loss and a power cut.

Team 10: one stage each

`Team Lead`

[Name]

Process: leak rules, night watch, valve and buzzer; integration

Demo: a leak closes the valve

`Second Lead`

[Name]

Ingest and Store: Wi-Fi, MQTT, HTTPS, Supabase

Demo: Last Will and the performance panel

`Member`

[Name]

Sense and hardware: wiring, sensors, calibration

Demo: the bucket test and pin map

`Member`

[Name]

Analyze: dashboard, five answers, report and slides

Demo: a week of answers

The split follows the firmware’s files, one per pipeline stage, so each member can explain their own part.

Find the leak, stop the water, see the cost.

Next: keep learning in flash · check TLS certificates · a private broker · phone alerts · more sensor pairs

sustainflow.netlify.app

github.com/Darshanpawar7/Sustain-x

Questions?